

Calculated columns with DAX

# DAX data types, operators, and variables

# DAX data types, operators, and variables

In DAX, we work with various **data types**, each tailored to specific **kinds of data**, such as numbers, text, and dates. It determines **how data can be manipulated** in calculations.

**Operators** in DAX, like arithmetic, comparison, string, and logical operators, allow us to **perform calculations** and **evaluate expressions**, shaping how we analyse and derive insights.

DAX **variables** enable us to **store intermediate results**, simplify complex expressions, and enhance formula readability and performance.

# DAX data types

Data types in DAX define the **kind of data** that can be stored in a column or used in expressions and calculations.

There are a few data types that we can use in DAX, but we will focus on the following:

## Whole number

Integer values without fractions.

## Decimal number

Real numbers with fractions.

## Boolean

True or false values.

## Text

Character strings.

## Date

Dates and times.

## Currency

Monetary values.

## Blank

Represents an absence of data.

# Using DAX with data

We will use the following Country\_stats dataset to illustrate how we use DAX.

| Country_ID | Region          | Is_landlocked | GDP_per_capita_USD | Population_size | Area_km2     | Survey_date |
|------------|-----------------|---------------|--------------------|-----------------|--------------|-------------|
| EGY        | Northern Africa | FALSE         | 4,295.40           | 110,990,103     | 1,010,408.45 | 2019-07-16  |
| ETH        | Eastern Africa  | TRUE          | 1,027.60           | 123,379,924     | 1,112,000.71 | 2022-07-28  |
| GHA        | Western Africa  | FALSE         | 2,175.90           | 33,475,870      | 239,535.52   | BLANK       |
| KEN        | Eastern Africa  | FALSE         | 2,099.30           | 54,027,487      | 580,367.59   | 2022-02-01  |
| MAR        | Northern Africa | FALSE         | 3,527.90           | 37,457,971      | 446,300.34   | 2021-06-21  |
| RWA        | Eastern Africa  | TRUE          | 966.3              | 13,776,698      | 26,338.50    | 2022-04-12  |
| NGA        | Western Africa  | FALSE         | 2,184.40           | 218,541,212     | 923,769.19   | 2020-12-11  |
| ZAF        | Southern Africa | FALSE         | 6,776.50           | 59,893,885      | 1,221,037.55 | 2019-07-19  |

# DAX data types

| Data type      | Description                                                                                                                                   | Example         |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Whole number   | Handles <b>integers</b> that <b>do not</b> have a decimal place. Used when decimals/fractions are not applicable or precision isn't required. | Population_size |
| Decimal number | Can handle numbers with <b>decimal places</b> and whole numbers. Needed for precise data.                                                     | Area_km2        |
| Boolean        | A value that is either <b>TRUE</b> or <b>FALSE</b> . Fundamental in creating filters or conditional statements in DAX.                        | Is_landlocked   |
| Text           | A <b>character</b> data string, which can be letters, numbers, or dates represented in a text format.                                         | Region          |

# DAX data types

| Data type | Description                                                                                                    | Example                                |
|-----------|----------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Date      | Represents a date, storing the day, month, and year.                                                           | Survey_date                            |
| Currency  | Represents monetary values. Designed for financial calculations where accuracy and rounding rules are crucial. | GDP_per_capita_USD                     |
| Blank     | Blank is a DAX data type that signifies an unknown or empty value. (It represents and replaces SQL nulls.)     | Any empty values (look at Survey_date) |

# Operators in DAX

Operators in DAX are **symbols** or **keywords** that are used to create expressions that compare values, perform arithmetic calculations, or work with strings.

Each operator type has a **specific function**, like arithmetic operators for mathematical calculations, comparison operators for filtering and sorting data, etc. Understanding operators is **crucial for building expressions** and **formulas**.



Think about the different data types in our Country\_stats dataset. Do you think we can use all the operators with all the different types of data? Why or why not?

# Arithmetic operators

Arithmetic operators perform **mathematical calculations** and **produce numeric results**. They allow us to create calculated columns or measures that involve summation, subtraction, or other mathematical transformations of data.

+

Used to **add** two **numbers**.

**Example:**

1,027.60 +  
2,175.90

-

Used to **subtract** one **number** from another.

**Example:**

580,367.59 -  
446,300.34

\*

Used for **multiplying** two **numbers**.

**Example:**

580,367.59 \* 3

/

Used to **divide** one **number** by another.

**Example:**

110,990,103 /  
123,379,924

^

Raises one number to the **power** of another.

**Example:**

239,535.52 ^ 2



# Comparison operators

Comparison operators in DAX are used to **compare values**. When two values are compared by using these operators, the **result** is a logical value, either **TRUE** or **FALSE**.

| <b>=</b>                                                               | <b>==</b>                                                                                                                    | <b>&gt;</b>                                                                        | <b>&gt;=</b>                                                                                   | <b>&lt;</b>                                                                     | <b>&lt;=</b>                                                                                | <b>&lt;&gt;</b>                                                                    |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Checks if<br><b>A = B</b>                                              | Checks if<br><b>A = B</b>                                                                                                    | Checks if<br><b>A &gt; B</b>                                                       | Checks if<br><b>A ≥ B</b>                                                                      | Checks if<br><b>A &lt; B</b>                                                    | Checks if<br><b>A ≤ B</b>                                                                   | Checks if<br><b>A ≠ B</b>                                                          |
| Returns TRUE if value A in a column <b>equals</b> a specified value B. | Returns TRUE if value A in a column and a specified B are <b>strictly equal</b> , i.e. have equal values or are both BLANK.* | Returns TRUE where value A in a column is <b>greater than</b> a specified value B. | Returns TRUE where value A in a column is <b>greater than or equal to</b> a specified value B. | Returns TRUE where value A in a column is <b>less than</b> a specified value B. | Returns TRUE where value A in a column is <b>less than or equal to</b> a specified value B. | Returns TRUE where value A in a column is <b>not equal</b> to a specified value B. |

\* = and == differ in handling BLANK: = returns TRUE when BLANK is compared to 0, an empty string, or FALSE; == returns TRUE only when BLANK is compared to BLANK.

# String operators

The primary string operator in DAX is the **concatenation** operator (&), which **joins** two or more text strings into a **single string**.

*This operator is especially useful in **creating calculated columns** where data from different text columns need to be combined for enhanced insights.*



We could create a new column in our dataset that combines the country's Country\_ID and Region into a single string, with a dash (-) between them.

**DAX:** Country\_region = [Country\_ID] & " - " & [Region]

| Country_region        |
|-----------------------|
| EGY - Northern Africa |
| ETH - Eastern Africa  |
| GHA - Western Africa  |
| KEN - Eastern Africa  |
| MAR - Northern Africa |
| RWA - Eastern Africa  |
| NGA - Western Africa  |
| ZAF - Southern Africa |

# Logical operators

| Logical operator        | Description                                                                                                                                                                         | Example                                                                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>&amp;&amp; (AND)</b> | Returns TRUE if <b>both</b> conditions are TRUE; otherwise it returns FALSE.                                                                                                        | <b>Check if a country is landlocked and has a population over 100 million:</b> <code>[Is_landlocked] &amp;&amp; [Population_size] &gt; 100000000</code> |
| <b>   (OR)</b>          | Returns TRUE if <b>at least one</b> of the expressions is TRUE; it's only FALSE if both expressions are FALSE.                                                                      | <b>Check if a country is in either Eastern Africa or Northern Africa:</b> <code>[Region] = "Eastern Africa"    [Region] = "Northern Africa"</code>      |
| <b>IN</b>               | Checks if a value <b>exists</b> within a <b>specified list</b> or <b>table</b> , returning TRUE if a match is found or FALSE otherwise.                                             | <b>Check if the country's region is in Northern Africa:</b> <code>[Region] IN {"Northern Africa"}</code>                                                |
| <b>NOT</b>              | <b>Reverses</b> the <b>logical value</b> of an expression, returning TRUE if the expression is FALSE, and vice versa. It is a unary operator, i.e. it operates on only one operand. | <b>Check if a country is not landlocked:</b> <code>NOT([Is_landlocked])</code>                                                                          |

# Operator precedence



|                                  |                             |
|----------------------------------|-----------------------------|
| $\wedge$                         | Exponentiation              |
| $-$                              | Sign (as in $-1$ )          |
| $*$ and $/$                      | Multiplication and division |
| $+$ and $-$                      | Addition and subtraction    |
| $\&$                             | Concatenation               |
| $=, <, >, <=, >=, <>, \text{IN}$ | Comparison                  |
| NOT                              | NOT (unary operator)        |



## Order of operators

If we combine several operators in a single formula, the operations are **ordered** according to the table. If the operators have equal precedence value, they are ordered from **left to right**.

*To change the order of evaluation, we can enclose the part of the formula that must be calculated first in parentheses.*

# Variables in DAX

Variables are used to **store** the **result** of an expression as a **named** value. Defining a variable involves **assigning** an expression to a **name**, which can then be **used** throughout a DAX formula.

## The need for variables in DAX

Variables are essential for **storing** and **reusing intermediate** results within expressions, eliminating the need for repeated calculations of the same expression within a formula.

Variables allow for a more **organised** and **efficient** way of handling complex calculations, thereby **simplifying** our DAX expressions and improving **performance**.

# Defining variables

## VAR and RETURN

To **define** a variable in DAX, we use the "**VAR**" keyword, followed by the **variable name** and the **expression** we want to assign to it. **The syntax is: VAR VariableName = Expression**

Once variables are defined, we use "**RETURN**" to **output** the formula's result, incorporating the defined variables.

Define a variable `Max_population` to store the maximum value from the `Population_size` column, and then return this value.



```
VAR Max_population = MAX([Population_size])  
RETURN  
Max_population
```

# Advantages of using variables

## Readability and clarity

Variables make DAX formulas **easier** to **read** and **understand**, especially when dealing with lengthy and complex expressions.

## Code reusability

Once defined, a variable can be **reused** multiple times within a formula, which helps in keeping the code **concise** and **manageable**.

## Optimise performance

By storing intermediate results, variables can **reduce** the **number** of calculations required, thus **optimising** the **performance** of the DAX expression.

## Easier debugging

Variables **simplify** the debugging process, as they allow us to **isolate** and **inspect** parts of the formula, making it easier to **identify** and fix errors